YouTube AI Agent

Engineering Challenges & Solutions
AIMultiple Technical Assessment | Development Retrospective | October 2026

01 Handling HTTP 403 responses from Genius

Challenge

Automated Genius requests occasionally returned HTTP 403, including during a Michael Jackson album run. Finding a valid song URL did not always mean that its lyrics could be retrieved.

Solution implemented

The workflow separates URL discovery (genius_finder.py) from extraction (genius_scraper.py). Apify-based retrieval, alternative candidate URLs, retry/fallback attempts, and a lyrics cache were added to make recoverable failures less disruptive. Failed requests are handled explicitly rather than silently accepted.

Observed example: Michael Jackson - Got to Be There; some requests required another attempt.

Engineering takeaway: External websites are not deterministic APIs; resilient pipelines need recovery paths and clear failure reporting.

02 Matching album tracks to Genius pages

Challenge

Track names in album metadata sometimes included featured-artist credits or formatting absent from Genius page titles. Exact-title matching could miss valid pages.

Solution implemented

The Genius finder generates normalized title candidates and checks potential Genius URLs. The original album track title is preserved in the output while simplified variants are used for lookup.

Observed example: "I Will Still Love You (with Don Philip)" was searched using the shorter candidate "I Will Still Love You".

Engineering takeaway: Normalize search inputs without changing the source metadata.

03 Cleaning lyrics and repairing text encoding

Challenge

Scraped pages could contain contributor text, section headings, irregular whitespace, and corrupted characters. Such differences affect word counts, token counts, and text hashes.

Solution implemented

The lyrics processor normalizes line endings, repairs common encoding artifacts, removes page metadata and section labels, and tokenizes the resulting text with tiktoken cl100k_base. A token encode/decode round-trip check helps confirm internal text consistency.

Example: malformed apostrophe sequences are repaired; headings such as [Chorus] and [Verse 1] are removed during cleaning.

Engineering takeaway: Text preparation must be deterministic. In the current implementation, lyrics MD5 is computed from cleaned text, so the hash represents the processed lyrics rather than the unmodified

scraped source.

04 Resuming interrupted album processing

Challenge

An album requires many external requests; a mid-run failure could otherwise force all tracks to be processed again.

Solution implemented

The album processor records per-track checkpoints and caches successfully retrieved lyrics. On subsequent runs, it validates the artist, album, and track list before reusing completed work, then continues with outstanding tracks.

Recorded example: Britney Spears debut album - 11/11 tracks processed, 6,249 total tokens.

Engineering takeaway: Checkpointing improves recoverability and reduces redundant network requests.

05 Reproducible token counts and embedding hashes

Challenge

Token counts depend on the tokenizer and exact text. The embedding-to-MD5 process additionally depends on track order, embedding model, and numerical serialization.

Solution implemented

The pipeline computes ordered cl100k_base counts, joins them with commas, generates a 768-dimensional embedding using nomic-ai/nomic-embed-text-v1.5, formats each value to ten decimal places, and calculates MD5 from the serialized UTF-8 text.

Recorded Madonna debut album: 8 tracks; counts 618, 587, 633, 458, 567, 613, 549, 888; total 4,913; mean 614.125. Local embedding hash: 2e5a2b9787d9516816da5c74162ad8f4.

Engineering takeaway: Explicit tokenizer, ordering, model, and formatting choices make outputs easier to reproduce; a recorded hash is environment-dependent.

06 Preparing a reviewable GitHub submission

Challenge

Development produced diagnostic scripts and intermediate artifacts alongside the application, making the repository harder to navigate.

Solution implemented

The submission was organized around main.py and five focused modules for album processing, Genius discovery, scraping, lyrics processing, and hashing. Development-only test scripts were removed from Git tracking while remaining available locally. The README, dependency file, environment template, and representative Madonna/Britney JSON outputs were retained.

Packaging approach: a modular CLI entry point with separate components for distinct responsibilities.

Engineering takeaway: A concise repository makes the execution path and architectural responsibilities easier for reviewers to inspect.

Summary

The project strengthened its handling of unreliable external services, inconsistent track metadata, text quality, interrupted execution, deterministic processing, and repository organization. Recorded local runs covered Madonna (8 tracks, 4,913 tokens) and Britney Spears (11 tracks, 6,249 tokens). These results document observed runs rather than claiming exhaustive validation across environments.